

TaskFlow

Full Stack Task Management System with Authentication & Analytics

Submitted by	Anshika Saklani
Institution	Graphic Era Hill University, Dehradun
Role	Full Stack Developer — Finflock Systems Private Limited
Date	March 21, 2026
Assignment	Full Stack Developer Assignment

FastAPI	PostgreSQL	React 18	TypeScript	Tailwind CSS	JWT Auth
---------	------------	----------	------------	--------------	----------

1. Project Overview

TaskFlow is a production-grade full stack task management system built as a Full Stack Developer Assignment at Finflock Systems Private Limited. It demonstrates structured backend architecture, secure JWT authentication, relational database design, and a polished analytics dashboard frontend.

Key Highlights

- Layered FastAPI backend: API / Service / Repository / Model / Schema
- Secure JWT with access token (30 min) + refresh token (7 days) rotation
- PostgreSQL schema — 4 normalized tables with FK constraints and cascade rules
- Automatic TaskHistory audit log appended on every task mutation
- Analytics engine: 8 required metrics + custom weighted productivity score
- React 18 + TypeScript SPA with protected routes and silent token refresh
- Recharts visualizations: area chart, pie chart, bar chart
- Docker + docker-compose for one-command local deployment
- Swagger/OpenAPI interactive docs at /api/docs

2. Technical Stack

Layer	Technology	Version	Purpose
Backend	FastAPI	0.111.0	REST API framework
Backend	SQLAlchemy	2.0.30	ORM / database layer
Backend	Alembic	1.13.1	Database migrations
Backend	Pydantic v2	2.7.1	Request/response validation
Backend	python-jose	3.3.0	JWT encoding/decoding
Backend	passlib[bcrypt]	1.7.4	Password hashing
Database	PostgreSQL	16	Primary relational database
Frontend	React	18.3.1	UI framework
Frontend	TypeScript	5.4.5	Type-safe JavaScript
Frontend	Vite	5.2.13	Build tool / dev server
Frontend	Tailwind CSS	3.4.4	Utility-first styling
Frontend	Zustand	4.5.2	Client state management
Frontend	Recharts	2.12.7	Data visualization
Frontend	React Hook Form	7.51.5	Form validation
Frontend	Axios	1.7.2	HTTP client with interceptors
DevOps	Docker + Compose	Latest	Containerized deployment

3. Backend Architecture

The backend follows a strict layered architecture with clear separation of concerns.

```
backend/  
  app/  
    api/routes/ # HTTP layer: auth, tasks, analytics, feedback  
    models/ # SQLAlchemy ORM models  
    schemas/ # Pydantic v2 request/response schemas  
    services/ # Business logic  
    repositories/ # Database query layer  
    core/ # DB session, JWT security, auth dependency  
    config/ # Pydantic-settings environment config  
    migrations/ # Alembic migration files  
    main.py # Uvicorn entrypoint
```

Layer Responsibilities

Layer	Responsibility
API Routes	Receive HTTP requests, validate via Pydantic, delegate to services
Services	Orchestrate business logic, call repositories, raise HTTP exceptions
Repositories	All SQL queries via SQLAlchemy ORM — no business logic
Models	SQLAlchemy declarative models mapping to DB tables
Schemas	Pydantic v2 models for input validation and response serialization
Core	Shared infrastructure: DB session, JWT utilities, auth dependency

4. JWT Authentication Flow

The application implements a dual-token JWT strategy with short-lived access tokens and long-lived refresh tokens, following industry best practices for stateless authentication.

Token Strategy

Token Type	Lifetime	Algorithm	Purpose
Access Token	30 minutes	HS256	Sent with every API request in Authorization header
Refresh Token	7 days	HS256	Used only to obtain a new token pair

Step-by-Step Flow

Step	Endpoint	Detail
1. Register	POST /api/auth/register	Password bcrypt-hashed. Returns access_token + refresh_token + user object
2. Login	POST /api/auth/login	Credentials verified against bcrypt hash. Returns new token pair.
3. API Request	Authorization: Bearer	Every protected endpoint validates JWT via get_current_user dependency. P
4. Token Refresh	POST /api/auth/refresh	Axios interceptor calls this on 401 automatically. Validates refresh type claim
5. Logout	Client-side	Both tokens removed from localStorage. User redirected to /login.

5. Database Design

The schema follows 3NF normalization with four core entities, proper FK constraints, cascade delete rules, and strategic indexes for query performance.

Entities

Table	Primary Key	Foreign Keys	Notes
users	id SERIAL	—	Unique index on email. Stores bcrypt-hashed passwords.
tasks	id SERIAL	user_id -> users CASCADE DELETE	Enum columns for priority and status. Indexed on user_id.
task_history	id SERIAL	task_id -> tasks CASCADE DELETE	Append-only audit log. previous_state stored as JSONB.
feedback	id SERIAL	user_id -> users CASCADE, task_id -> tasks SET NULL	task_id is SET NULL. Feedback can exist without a task.

Indexing Strategy

Index	Type	Justification
users.email	UNIQUE INDEX	Login lookup — must be O(1). Uniqueness enforced at DB level.
tasks.user_id	INDEX	All task queries filter by user_id. Prevents full table scans.
task_history.task_id	INDEX	History queries filter by task_id. Fast audit log retrieval.
feedback.user_id	INDEX	Feedback queries filter by user.
feedback.task_id	INDEX	Join from tasks to feedback. Nullable FK still benefits from index.

6. API Endpoints

All endpoints are documented interactively at <http://localhost:8000/api/docs> (Swagger UI). Protected endpoints require Authorization: Bearer token.

Method	Endpoint	Auth	Description
POST	/api/auth/register	No	Register new user, returns token pair
POST	/api/auth/login	No	Login, returns token pair
POST	/api/auth/refresh	No	Exchange refresh token for new token pair
GET	/api/tasks	Yes	List tasks — paginated, filterable by priority/status/date
POST	/api/tasks	Yes	Create a new task
GET	/api/tasks/{id}	Yes	Get single task by ID
PUT	/api/tasks/{id}	Yes	Update task fields
DELETE	/api/tasks/{id}	Yes	Delete task (creates history record first)
GET	/api/tasks/{id}/history	Yes	Get full audit history for a task
GET	/api/analytics	Yes	Get all analytics data for dashboard
POST	/api/feedback	Yes	Submit feedback with 1-5 star rating
GET	/api/feedback	Yes	Get current user's feedback list
GET	/api/health	No	Health check endpoint

7. Analytics Engine

The analytics service (app/services/analytics_service.py) computes all metrics server-side. Calculations are performed in Python after fetching the user's tasks from PostgreSQL.

Metrics Catalogue

Metric	Calculation
Total Tasks	COUNT all tasks for user
Completed Tasks	COUNT WHERE status = Completed
Completion Percentage	(completed / total) x 100, 1 decimal
Completed Today	Filter WHERE updated_at.date() == today
Completed This Week	Filter WHERE updated_at >= now - 7 days
Completed This Month	Filter WHERE updated_at >= first day of month
Most Productive Day	Day with highest completed task count
Avg Completion Time	Mean of (updated_at - created_at) in hours
Priority Distribution	COUNT grouped by Low / Medium / High
Daily Chart Data	Last 30 days mapped to daily completion count
Productivity Score	Weighted formula — see below

Productivity Score Formula

Score = (completion_rate x 50) + (high_priority_rate x 30) + (on_time_rate x 20)

Component	Weight	Definition	Rationale
Completion Rate	50%	completed / total	Foundation — are you finishing tasks?
High Priority Rate	30%	high_completed / high_total	Rewards focus on what matters most
On-Time Rate	20%	done_before_due / tasks_with_due	Reincentivises deadline discipline. Defaults 0.5 if no due d

8. Frontend Architecture

Single-page application built with React 18, TypeScript, and Vite. Clean separation between UI components, service layer, state management, and routing.

Pages

Page	Route	Description
Login	/login	Email/password form with error handling
Register	/register	New account creation with validation
Dashboard	/dashboard	Analytics overview: stat cards and charts
Tasks	/tasks	Full task management: filters, search, pagination
Analytics	/analytics	Deep-dive charts: area, bar, pie + metrics

State Management

Zustand manages two stores: `auth.store.ts` (`user`, `isAuthenticated`, `setAuth`, `logout`) and `task.store.ts` (`tasks`, `filters`, `pagination`, `loading`). The `Axios api.ts` instance has a request interceptor (attach Bearer token) and a response interceptor (silent token refresh on 401).

Key Features

- Protected routes — unauthenticated users redirect to `/login`
- Silent token refresh — fully transparent to user
- Task history modal — full audit timeline per task
- Client-side search + server-side priority/status filters
- Pagination with configurable page size
- Loading spinners and error messages on all async operations
- Responsive dark-themed UI with Tailwind CSS
- Form validation via React Hook Form on all forms

9. Local Setup Instructions

Prerequisites

Python 3.11.9 | Node.js 20+ | PostgreSQL 16

Step 1 — Create Database

```
psql -U postgres -c "CREATE DATABASE taskflow;"
```

Step 2 — Backend

```
cd backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
# Edit .env with your DATABASE_URL password
py main.py # http://localhost:8000
```

Step 3 — Frontend

```
cd frontend
npm install
npm run dev # http://localhost:5173
```

Step 4 — Docker (Alternative)

```
docker-compose up --build
```

Environment Variables

Variable	Default	Description
DATABASE_URL	postgresql://postgres:pwd@localhost:5432/taskflow	PostgreSQL connection string
SECRET_KEY	(change in production)	JWT signing secret, min 32 chars
ALGORITHM	HS256	JWT signing algorithm
ACCESS_TOKEN_EXPIRE_MINUTES	30	Access token lifetime in minutes
REFRESH_TOKEN_EXPIRE_DAYS	7	Refresh token lifetime in days
BACKEND_CORS_ORIGINS	["http://localhost:5173"]	Allowed CORS origins

10. Evaluation Criteria Checklist

Criteria	Weight	Implementation	Status
Backend architecture	20%	FastAPI: API/Service/Repository/Model/Schema layers	Done
JWT implementation	15%	Dual-token, bcrypt, type claim, silent refresh	Done
Database design	15%	4 normalized tables, FK constraints, indexes	Done
Analytics logic	10%	8 metrics + weighted productivity score formula	Done
Frontend architecture	15%	React 18 + TypeScript, protected routes, Zustand	Done
UI/UX quality	10%	Dark theme, Tailwind, responsive, loading/error states	Done
Code quality	10%	Type-safe, no duplication, consistent patterns	Done
Documentation	5%	README + Swagger + ER diagram + PDF report	Done

Bonus Features Implemented

- Docker + docker-compose for one-command deployment
- Full Swagger/OpenAPI interactive documentation at /api/docs
- ER Diagram (er_diagram.svg) in project root
- Task history modal with full audit timeline per task
- Feedback system with 1-5 star ratings